STANDARD TEMPLATE LIBRARY-STL

Dr. S. Vinila Jinny

Definition

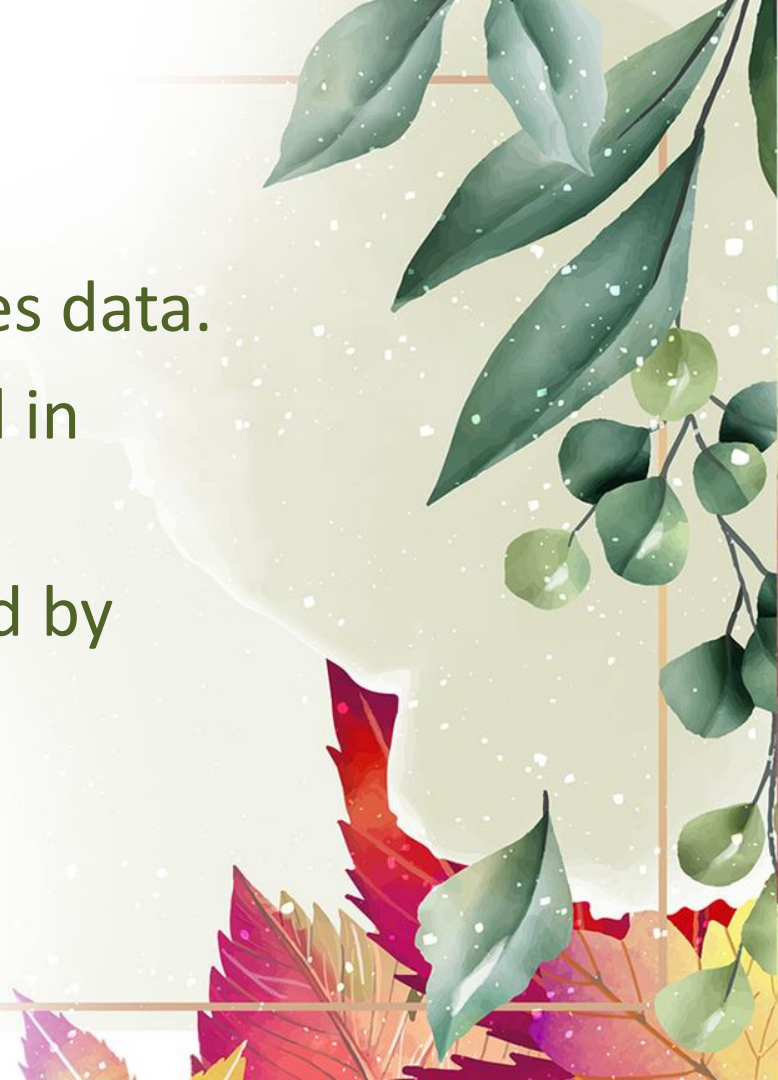
- Set of C++ template classes and functions
- Generic Classes and Functions
- Used to implement data structures and algorithms
- Used for storing and processing of data
- All are defined in namespace std directive

Components of STL

- Containers
- Algorithms
- Iterators
  - Algorithm employ iterators to perform operations stored in containers.

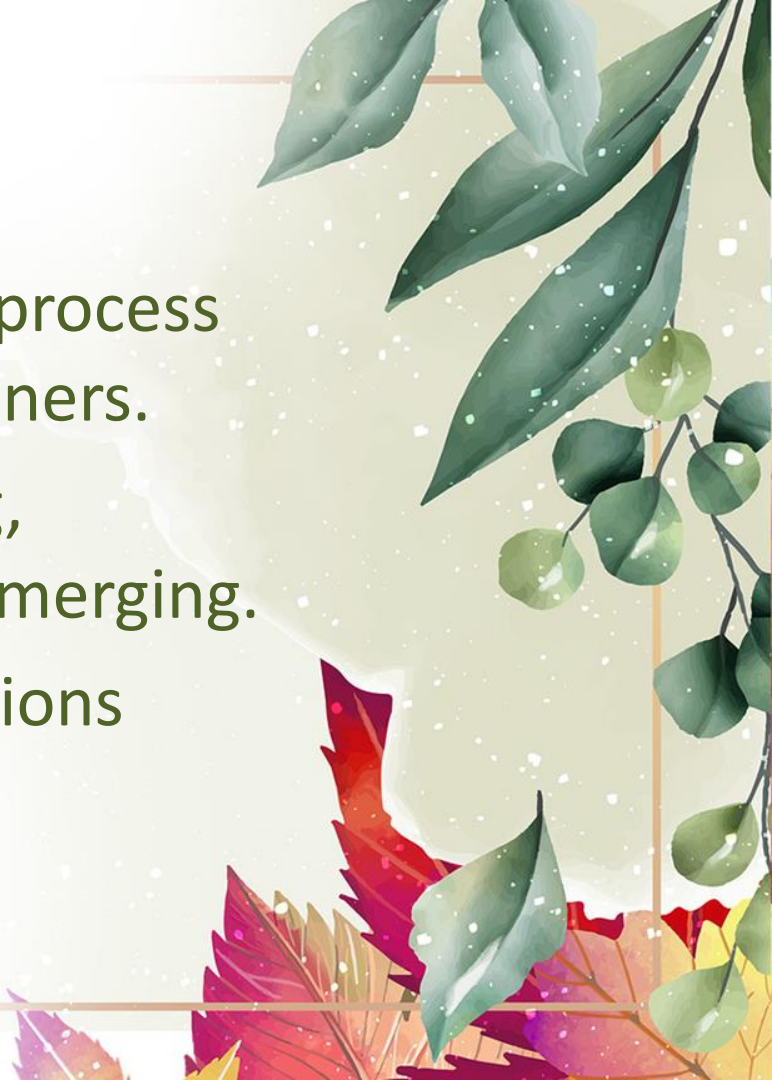
Container

- It is an object that actually stores data.
- It is a way that data is organized in memory.
- STL Containers are implemented by template classes



Algorithms

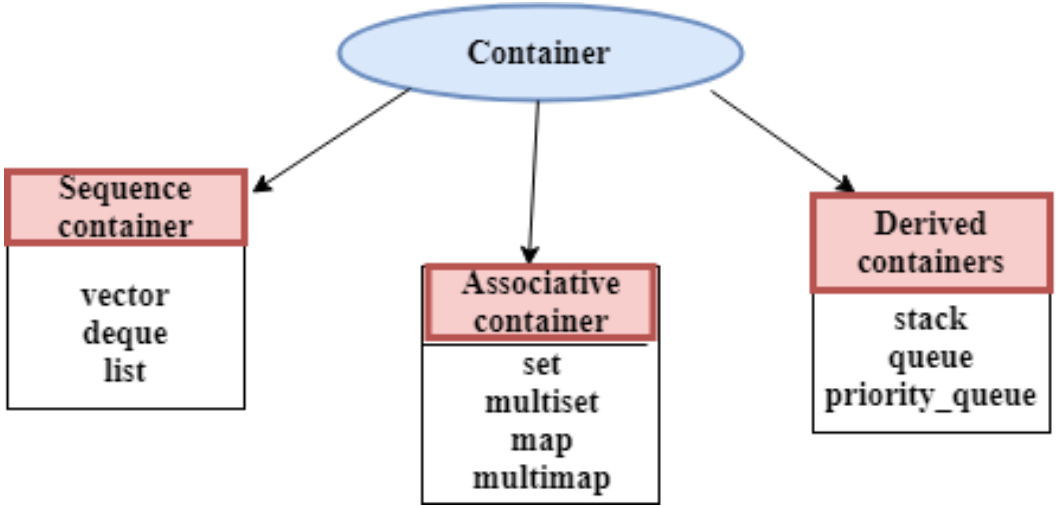
- It is a procedure that is used to process the data contained in the containers.
- STL support tasks like initializing, searching, copying, sorting and merging.
- Implemented by template functions



Iterator

- It is an object(like a pointer) that points to an element in a container.
- Iterators are used to move through the contents of containers
- Iterators are handled just like pointers
- Iterators connect algorithms with containers and can manipulate the data stored in the container.

STL has 10 Containers under 3 category



Containers are objects that hold data(of same datatype)

Containers Supported by STL

container	Description	Header file	iterator
vector	vector is a class that creates a dynamic array allowing insertions and deletions at the back.	<vector>	Random access
list	list is the sequence containers that allow the insertions and deletions from anywhere.	<list>	Bidirectional
deque	deque is the double ended queue that allows the insertion and deletion from both the ends.	<deque>	Random access
set	set is an associate container for storing unique sets.	<set>	Bidirectional
multiset	Multiset is an associate container for storing non- unique sets.	<set>	Bidirectional

Containers Supported by STL

container	Description	Header file	iterator
map	Map is an associate container for storing unique key-value pairs, i.e. each key is associated with only one value(one to one mapping).	<map>	Bidirectional
multimap	multimap is an associate container for storing key- value pair, and each key can be associated with more than one value.	<map>	Bidirectional
stack	It follows last in first out(LIFO).	<stack>	No iterator
queue	It follows first in first out(FIFO).	<queue>	No iterator
Priority-queue	First element out is always the highest priority element.	<queue>	No iterator

Category of Container

- **Sequence containers: These are used to implement sequential data structures like a linked list, array, etc.**
- **Associative containers: These are those containers in which each element has a value that is related to a key. They are used to implement sorted data structures, for example, set, multiset, map, etc.**
- **Derived Containers: Derived Containers can be defined as an interface used to provide functionality to the pre-existing containers.**

Sequence Container

- **Vectors: Vectors can be defined as a dynamic array or an array with some additional features.**

Syntax: vector<int> v;

- **Deque: Deque is also known as a double-ended queue that allows inserting and deleting from both ends; they are more efficient than vectors in case of insertion and deletion.**

Syntax: deque <int> d;

- **List: List is also the sequential container and allows non-contiguous allocation. It allows insertion and deletion anywhere in the sequence.**

Syntax: list<int> l;

Sequence Container

- Elements are accessed using an iterator
- Three sequence container varies in performance(speed of access)

Associative Container

- Designed to support direct access to elements using keys.
- Not sequential
- Four types
  - Set
  - Multiset
  - Map
  - Multimap

Associative Container

- Stores data in a structure called tree which facilitates fast searching, deletion, insertion.
- Slow for random access and inefficient for sorting
- Difference between set and multiset is that a multiset allows duplicate items.
- Difference between map and multimap is that a map allows only one key for a value while multimap permits multiple keys.

Derived Containers

- Also called container adaptors
- stack, queue and priority queue can be created from different sequence containers
- Derived containers do not support iterators, so data manipulation not possible
- Has two member functions pop(), push() for performing deletion and insertion

Iterators

- Iterators are constructs that we use to traverse or step through containers in STL.
- Iterators are very important in STL as they act as a bridge between algorithms and containers.
- Iterators always point to containers and in fact algorithms actually, operate on iterators and never directly on containers.

Types of Iterators

- **Input Iterators:** Simplest and is used mostly in single-pass algorithms.
- **Output Iterators:** Same as input iterators but not used for traversing.
- **Bidirectional Iterators:** These iterators can move in both directions.
- **Forward Iterators:** Can be used only in the forward direction, one step at a time.
- **Random Access Iterators:** Same as pointers. Can be used to access any element randomly.

Algorithms

- Algorithms are a set of functions or methods provided by STL that act on containers.
- These are built-in functions and can be used directly with the STL containers and iterators instead of writing our own algorithms.
- More than 60 algorithms available
- STL algorithms are not member function or friend functions but standalone template function.
- To use this include <alogrithm> in our program.

Types of Algorithms

- Searching algorithms
- Sorting algorithms
- Modifying or manipulating algorithms
- Non-modifying algorithms
- Numeric algorithms
- Min/Max algorithms

Vector

- Widely used container
- Stores elements in contiguous memory
- Enables direct access to element using subscript [] operator.
- Can change its size dynamically and therefore allocates memory as needed at run time.

Vector-definition

- A vector is a sequence container class that implements dynamic array, means size automatically changes when appending elements.
- A vector stores the elements in contiguous memory locations and allocates the memory as needed at run time.

Vector

- Supports random access iterators
- Class vector supports a number of constructors for creating objects.
  - **vector<int> v1;//vector of size 0**
  - **vector<double> v2(10)//vector of size 10-double type**
  - **vector<int> v3(v4)//vector created from another vector**
  - **vector<int> v(5,2)//vector of size 5 and values are 2**

Vector

- Support several member function and can use STL algorithms.

Vector Functions

Function	Description
at()	It provides a reference to an element.
back()	It gives a reference to the last element.
front()	It gives a reference to the first element.
swap()	It exchanges the elements between two vectors.
push_back()	It adds a new element at the end.
pop_back()	It removes a last element from the vector.
empty()	It determines whether the vector is empty or not.

Vector Functions

Function	Description
insert()	It inserts new element at the specified position.
erase()	It deletes the specified element.
resize()	It modifies the size of the vector.
clear()	It removes all the elements from the vector.
size()	It determines a number of elements in the vector.
capacity()	It determines the current capacity of the vector.
assign()	It assigns new values to the vector.

Vector Functions

Function	Description
operator=()	It assigns new values to the vector container.
operator[]()	It access a specified element.
end()	It refers to the past-lats-element in the vector.
emplace()	It inserts a new element just before the position pos.
emplace_back()	It inserts a new element at the end.
rend()	It points the element preceding the first element of the vector.
rbegin()	It points the last element of the vector.

Vector Functions

Function	Description
begin()	It points the first element of the vector.
max_size()	It determines the maximum size that vector can hold.
cend()	It refers to the past-last-element in the vector.
cbegin()	It refers to the first element of the vector.
crbegin()	It refers to the last character of the vector.
crend()	It refers to the element preceding the first element of the vector.
data()	It writes the data of the vector into an array.
shrink_to_fit()	It reduces the capacity and makes it equal to the size of the vector.

Vector

```
#include <iostream>
#include <vector>
#include <iterator>
using namespace std;
void display(vector <int> &v)
{
for(int i=0;i<v.size();i++)
{
cout<<v[i]<<" ";
cout<<"\n";
}}
int main()
{
  vector<int> vec;
  int i;
  cout << "vector size = " << vec.size() <<
endl;
for(i = 0; i < 5; i++)
   {
    vec.push_back(i);
   }
   cout << "extended vector size = " << vec.size() << endl;
   cout<<"Current contents=";
   display(vec);
   vector<int>::iterator myv=vec.begin();
   while( myv != vec.end())
      {
       cout << "value of v = " << *myv << endl;
        myv++;
      }
   vec.insert(vec.begin(),3,9);
   cout<<"After Insertion:";
   display(vec);
   vec.erase(vec.begin()+3,vec.begin()+5);
   cout<<"After Deletion:";
   display(vec);
   return 0;
}
```